

JALRAKSHAK AI

Backend + Platform Engineering

Phase-wise AI Implementation Prompts, Credential Map, Security Rules and Build Checklist

SMART INDIA HACKATHON 2026

Problem Statement: SIH26071

Theme: Disaster Management

Team: NeuroBots

Purpose: Zero-to-build backend guide for AI coding assistants / IDE agents

HOW TO USE THIS PDF

Attach the JalRakshak Backend + Platform PRD and the full JalRakshak PRD to your coding assistant. Paste the Master Context once. Then execute Phase 0 through Phase 5 in order. Do not let the agent automatically skip phases. Each phase must finish with tests, migrations, boot verification and a short acceptance report.

1. Backend Delivery Strategy

The backend is the public trust boundary and integration hub for JalRakshak AI. The design intentionally starts as a modular monolith with a few separate compute/tile services. This keeps the SIH build realistic while preserving a clean path to future scaling.

- Backend exposes one stable /api/v1 boundary to the Web Command Centre and Mobile App.
- ML remains an internal service accessed through MLProvider; web/mobile never call ML directly.
- PostgreSQL + PostGIS is the system of record for operational and geospatial entities.
- Large weather/model artifacts stay in S3-compatible object storage, not relational BLOB columns.
- Redis + Celery handles early async jobs; Redis Streams provides durable event history; Redpanda is an optional scale path.
- Every external integration is wrapped behind an interface and must have a local/stub fallback.

2. Implementation Phases

Phase	Focus	Main Deliverable
Phase 0	Foundation	Contracts, repo, Compose, DB, providers, auth skeleton
Phase 1	Core Data + GIS	Risk/weather APIs, PostGIS, tiles, incidents, reports, realtime
Phase 2	Ingestion + Alerts	Weather adapters, async ML orchestration, outbox, CAP, push notifications
Phase 3	Real ML + Field Ops	ML client, report verification, routing, responder tasks, offline sync
Phase 4	Replay + Security	Historical replay, OR-Tools, audit, security hardening
Phase 5	Production Hardening	Observability, performance, CI, deployment, final documentation

3. Master Context Prompt - Paste Once

You are the principal backend/platform engineer for JalRakshak AI - SIH26071.

I have attached:

1. The complete JalRakshak AI PRD.
2. The dedicated Backend + Platform Engineering guide.

Read both documents completely before modifying any code.

We are implementing the FULL backend scope, not only the MVP or "must-build" features. However, implementation must be phased so every phase leaves the repository in a working state.

JalRakshak consists of four independent teams:

- ML + Geospatial
- Backend + Platform
- Web Command Centre
- Mobile App

You own ONLY Backend + Platform. Do not implement ML models, web UI or mobile UI. Integrate them only through contracts/providers.

Architectural rules:

- FastAPI + Python 3.12.
- Modular monolith initially, with independent compute/tile services where justified.
- PostgreSQL + PostGIS is the system of record.
- TimescaleDB or partitioned PostgreSQL for temporal weather/risk history.
- Redis + Celery initially.
- Redis Streams initially; Redpanda-compatible abstraction for future scaling.
- MinIO/S3-compatible object storage for rasters, COG, Zarr, media and replay artifacts.
- TiTiler for raster tiles.
- pg_tileserv/PostGIS MVT for vector tiles.
- Supabase Auth/OIDC for MVP authentication.
- OpenTelemetry + Prometheus + Grafana + Sentry for observability.
- Docker Compose must run the complete local stack.

The backend is the only public trust boundary. Web/mobile must never communicate directly with ML or databases.

Freeze contracts before features:

- /api/v1
- OpenAPI
- internal ML OpenAPI
- WebSocket event schemas
- GeoJSON/H3 conventions
- enums
- error envelope
- mock fixtures

Canonical conventions:

- timestamps: UTC ISO-8601
- API CRS: EPSG:4326
- rainfall rate: mm/h
- rainfall accumulation: mm
- water depth: metres only when calibrated
- risk enum: LOW, MODERATE, HIGH, SEVERE
- confidence: 0.0-1.0
- spatial key: H3 cell + optional admin/ward ID
- rasters: COG for serving, Zarr/NetCDF for compute
- model results always contain model_version and data_version
- errors: {code, message, trace_id, details}

Build providers behind interfaces:

- MLProvider
- NotificationProvider
- RoutingProvider
- ObjectStoreProvider

Every provider must have a deterministic local/stub implementation so development never stops because another team or third-party credential is unavailable.

SECURITY RULE:

Never hard-code, print, commit, log or expose credentials. Create .env.example containing variable NAMES and placeholders only. .env must be gitignored. If a phase requires an external credential, stop only for that credential and clearly tell me:

1. service name,
2. exact dashboard/page where I obtain it,
3. environment-variable name,
4. whether it is public or secret,

5. whether that phase can continue using a local/stub provider.

Never ask me to paste a secret into source code or commit history.

Before implementing each phase:

1. inspect existing repository,
2. tell me what already exists,
3. state files that will change,
4. state migrations/services that will be added.

After each phase:

- run lint/type checks,
- run tests,
- run migrations,
- boot Docker Compose where relevant,
- test endpoints,
- show completed acceptance criteria,
- show unresolved issues,
- DO NOT automatically begin the next phase until I tell you to continue.

4. Credential and Tool Map

Most of the backend stack is local/self-hosted and does not need paid API keys. External credentials should be introduced only when the corresponding integration is actually enabled.

Service	Credential?	Where it comes from	Environment variables	Sensitivity	Can continue without it?
PostgreSQL/PostGIS (local)	No external account	Choose local username/password in .env	POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, DATABASE_URL	Secret	Yes
Redis (local)	No external account	Docker/local config; password optional for dev	REDIS_URL, optional REDIS_PASSWORD	Secret if set	Yes
MinIO (local)	No external account	Choose credentials in Docker Compose/.env	MINIO_ROOT_USER, MINIO_ROOT_PASSWORD or app access keys	Secret	Yes
Supabase Auth	Yes	Supabase project dashboard -> API Keys / Connect	SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, optional SUPABASE_SECRET_KEY	Publishable + Secret	Yes, use auth stub/dev mode
ML internal service	Team internal	ML team provides service URL/token or generate shared internal token	ML_SERVICE_URL, ML_SERVICE_TOKEN	Secret	Yes, StubMLProvider
TiTiler	No	Self-hosted service; object-store credentials inherited	TITILER_URL	Not secret	Yes
pg_tileserv	No	Self-hosted; uses DB connection	PGTILESERV_URL	Not secret	Yes
Expo Push	Optional external	Expo/EAS only if enhanced push security enabled; device token comes from app	EXPO_PUSH_ACCESS_TOKEN	Secret	Yes, ConsoleNotificationProvider
CAP 1.2	No	Pure standard serialization	None	N/A	Yes
NDMA SACHET live publishing	Institutional authorization	Official government integration only	Institution-specific	Secret	Use CAP file/stub only
OSMnx / NetworkX	No	No key for prototype routing	None	N/A	Yes

OSRM / Valhalla self-hosted	No vendor key	Run service locally/server-side	ROUTING_SERVICE_URL	Not secret	Yes
OR-Tools	No	Python library	None	N/A	Yes
MLflow local	No	Local tracking server	MLFLOW_TRACKING_URI	Not secret locally	Yes
Prometheus	No	Local service	Config only	N/A	Yes
Grafana local	No external account	Choose local admin login	GRAFANA_ADMIN_USERNAME, GRAFANA_ADMIN_PASSWORD	Secret	Yes
Sentry	Yes if enabled	Sentry project -> Project Settings -> Client Keys (DSN)	SENTRY_DSN	Sensitive config	Yes, leave disabled
Redpanda local	No initially	Docker Compose	REDPANDA_BROKERS and optional auth vars	Depends	Yes
GitHub Actions	GitHub repo	Repository Settings -> Secrets and variables -> Actions	Deployment-specific secrets	Secret	Yes locally

CREDENTIAL SAFETY RULE

Do not paste real secrets into source files, prompts, screenshots or Git history. The coding assistant only needs the environment-variable names. You place actual values in your local .env or deployment secret manager. Commit .env.example only.

PHASE 0 - Foundation, Contracts and Local Platform

Goal: backend starts, local infrastructure is reproducible, contracts are frozen, database migrations work and every external dependency has a stub.

Copy/Paste Prompt

```
## PHASE 0 - FOUNDATION, CONTRACTS AND LOCAL PLATFORM
```

```
Implement Phase 0 only.
```

```
### A. Repository bootstrap
```

```
Establish:
```

```
backend/  
pyproject.toml  
app/  
  main.py  
  core/  
    config.py  
    security.py  
    errors.py  
    telemetry.py  
api/v1/  
domains/  
  weather/  
  risk/  
  incidents/  
  reports/  
  alerts/  
  routing/  
  assets/  
  responders/  
  replay/  
  audit/  
  models/  
integrations/  
  ml/  
  object_store/  
  notifications/  
  cap/  
  tiles/  
db/  
workers/  
realtime/  
migrations/  
tests/  
docker/
```

```
Use uv + pyproject.toml.
```

```
Add Ruff, mypy/pyright where appropriate, pytest, pytest-asyncio and pre-commit.
```

```
### B. Contracts
```

```
Establish/finalize:
```

```
contracts/
```

```

openapi.yaml
internal-ml-openapi.yaml
websocket-events.json
schemas/
fixtures/demo-event/

```

Define all current public endpoint paths even if some initially return stub data.

Define shared schemas:

- risk cell
- weather source
- nowcast manifest
- inundation manifest
- incident
- field report
- alert
- critical asset
- responder task
- model run
- audit event
- offline sync

C. Docker Compose infrastructure

Add:

- FastAPI
- PostgreSQL + PostGIS
- Redis
- MinIO
- Celery worker
- Celery scheduler
- ML stub

Do not add Kubernetes.

D. Database foundation

Install SQLAlchemy 2 async + Alembic.

Create first migrations including PostGIS and UUID support where required.

Base tables:

users/profiles, weather_sources, model_runs, risk_cells, incidents, field_reports, alerts, critical_assets, responder_tasks, audit_log, outbox_events.

Use indexes appropriate for geometry/GIST, H3, time, active incidents and valid risk snapshots.

E. Provider architecture

Create interfaces:

```

MLProvider
NotificationProvider
RoutingProvider
ObjectStoreProvider

```

Implement:

```

StubMLProvider
ConsoleNotificationProvider
StubRoutingProvider
MinIOObjectStoreProvider

```

Stub ML must deterministically provide rainfall manifest, inundation manifest, H3 risk cells and report verification result.

F. Authentication skeleton

Implement OIDC/JWT middleware compatible with Supabase Auth.

Roles:

CITIZEN

FIELD_RESPONDER

ANALYST

ALERT_APPROVER

MUNICIPAL_OFFICER

ADMIN

ML_ADMIN

Do not rely on frontend role hiding.

G. Initial API

Implement at minimum:

GET /health

GET /ready

GET /api/v1/weather/sources/status

GET /api/v1/models/status

Add /docs OpenAPI.

H. Configuration and secrets

Create typed Pydantic Settings configuration.

Create .env.example, not .env with real values.

Include placeholders for:

APP_ENV=

APP_SECRET=

DATABASE_URL=

POSTGRES_DB=

POSTGRES_USER=

POSTGRES_PASSWORD=

REDIS_URL=

MINIO_ENDPOINT=

MINIO_ACCESS_KEY=

MINIO_SECRET_KEY=

MINIO_BUCKET=

SUPABASE_URL=

SUPABASE_PUBLISHABLE_KEY=

SUPABASE_SECRET_KEY=

ML_SERVICE_URL=

ML_SERVICE_TOKEN=

LOG_LEVEL=

Generate local development credentials where we control the service. Never fabricate external-service credentials.

For Supabase, tell me exactly what I need and where I get it before requiring it.

Acceptance criteria

- docker compose up starts the stack.
- Alembic migration succeeds on a fresh DB.
- /health and /ready succeed.
- MinIO bucket initialization succeeds.
- Redis connectivity succeeds.
- Stub ML returns deterministic fixture.
- OpenAPI generation succeeds.
- Contract tests succeed.

Finish Phase 0 and stop.

Phase 0 credential checkpoint

- No paid/external credential is required to finish the local phase.
- Create local PostgreSQL, Redis and MinIO credentials yourself and keep them in .env only.
- Supabase can remain stubbed until real login testing. When enabled, obtain the project URL and keys from the Supabase project dashboard.
- Use a deterministic ML stub so backend development does not wait for the ML subteam.

PHASE 1 - Core Domain, Geospatial Serving and Live State

Goal: real operational data structures, geospatial delivery, incident lifecycle, signed report upload and live WebSocket state.

Copy/Paste Prompt

```
## PHASE 1 - CORE DOMAIN, GEOSPATIAL SERVING AND LIVE STATE
```

Continue from the verified Phase 0 repository. Do not rewrite Phase 0 architecture.

```
### Implement persistence properly
```

Complete tables/domain repositories for:

- weather_sources
- risk_cells
- incidents
- incident_events
- field_reports
- critical_assets
- model_runs

Add temporal history and proper indexes.

Risk cells must support:

- H3 cell
- valid_time
- probability
- confidence
- risk level
- geometry
- model_version
- data_version

```
### Build weather/risk APIs
```

Implement:

- GET /api/v1/weather/current
- GET /api/v1/weather/sources/status
- GET /api/v1/nowcast/manifest
- GET /api/v1/inundation/manifest
- GET /api/v1/risk/cells
- GET /api/v1/risk/{h3_cell}/timeline
- GET /api/v1/assets
- GET /api/v1/models/status

Support bbox filters, valid_time, pagination where appropriate and ETag/version semantics.

Every model-derived response must include model_version, data_version, valid_time and confidence.

```
### Geospatial serving
```

Implement raster route:

COG in MinIO -> TiTiler -> XYZ/TileJSON.

Implement vector route:

PostGIS -> pg_tileserv/ST_AsMVT -> MVT.

Never return massive million-cell GeoJSON.

Configure short caching for live layers and long caching for immutable replay assets.

```
### Incident state machine
```

Implement:

DETECTED -> OPEN -> ACKNOWLEDGED -> MITIGATING -> RESOLVED -> CLOSED

plus DISMISSED with mandatory reason.

Create append-only incident timeline.

Field reports

Implement draft field-report creation.

Add signed object-upload flow:

backend creates upload intent -> returns pre-signed URL -> client uploads directly to MinIO -> backend finalizes checksum/metadata -> ML verification job queued.

Never proxy large media through the FastAPI process unnecessarily.

Realtime

Implement WS /api/v1/live.

Initial events:

risk.cell.updated

incident.created

incident.updated

report.received

report.verified

source.degraded

model.run.completed

Add heartbeat, reconnect semantics and last_event_id.

Tests

Add PostGIS integration tests, signed-upload test, incident transition tests, H3/bbox tests, WebSocket authorization tests and tile manifest tests.

Run full regression and stop after Phase 1.

Phase 1 credential checkpoint

- TiTiler and pg_tileserv are self-hosted; no vendor API key is required.
- TiTiler needs access to the object store; pg_tileserv needs database connectivity.
- Keep private operational map layers behind auth/tokenized tile access.

PHASE 2 - Ingestion, Async Processing, Alerts and Notifications

Goal: ingestion adapters, reliable async model runs, CAP-compatible alert lifecycle, watch locations and notification providers.

Copy/Paste Prompt

```
## PHASE 2 - INGESTION, ASYNC PROCESSING, ALERTS AND NOTIFICATIONS
```

Continue from Phases 0-1.

```
### Weather source adapter architecture
```

Implement interface:

```
list_available()
```

```
fetch()
```

```
parse_metadata()
```

```
normalize()
```

```
health_check()
```

Implement first: FileSystemDemoAdapter.

It must replay frozen historical/demo weather data completely offline.

Add replaceable adapters/placeholders for radar, satellite, AWS/rain gauge and NWP.

Do not invent official API credentials.

Raw source object must be immutable.

Store raw artifact, checksum, source timestamp, ingestion timestamp, provenance, normalized artifact and manifest.

Missing radar/AWS must generate source.degraded, not crash the pipeline.

```
### ML orchestration
```

Implement model-run lifecycle:

create run -> background job -> call MLProvider -> store artifacts -> materialize risk -> emit domain event.

Add idempotency key based on issue_time + model_version + data_manifest.

Duplicate run requests must not duplicate output.

Use StubMLProvider for current tests.

```
### Celery
```

Use Celery for:

```
- ingestion
```

```
- ML orchestration
```

```
- report verification
```

```
- alert fan-out
```

```
- replay preprocessing
```

Use Redis broker initially.

```
### Reliable events
```

Implement database outbox pattern for critical events.

Add Redis Streams event adapter while preserving an interface suitable for Redpanda later.

```
### Alert domain
```

Implement:

AI/rule recommendation -> ANALYST draft -> ALERT_APPROVER approve -> publish -> fan-out -> delivery receipt.

API:

POST /api/v1/alerts/draft

POST /api/v1/alerts/{id}/approve

POST /api/v1/alerts/{id}/publish

Validate approver region, severity, geometry/H3, effective/expiry time, source risk snapshot, content and confidence.

CAP 1.2

Build an isolated CAP serializer.

Generate valid CAP-compatible XML/payload.

This is serialization only.

Do NOT claim or implement unauthorized publishing directly to NDMA SACHET.

Watch locations

Implement Home/Family/College/Office-style location records.

Associate with H3/admin zones.

Trigger notification when risk threshold crosses OR official alert intersects watched location.

Notification provider

Keep ConsoleNotificationProvider.

Add ExpoPushNotificationProvider.

Receive and store ExpoPushToken securely per device.

Add delivery receipts/retry handling.

If Expo enhanced push security is enabled, read server-side access token only from environment.

Do not hardcode SMS or WhatsApp providers.

Tests

idempotent ML run

outbox reliability

CAP serialization

region alert authorization

notification retry

watch-zone intersection

source degradation

Stop at end of Phase 2.

Phase 2 credential checkpoint

- CAP serialization needs no API key.
- Do not use a fake SACHET key. Real dissemination is institution-authorized; demo by exporting valid CAP-compatible payloads.
- Expo Push can work with device ExpoPushToken from the mobile app. A server access token is optional only when enhanced push security is enabled.
- Official IMD/MOSDAC data adapters should initially support files/historical datasets unless you have legitimately provided authorized access.

PHASE 3 - Real ML Integration, Routing, Responder and Offline Support

Goal: replace the ML stub with a real internal client, complete field-report verification, flood-aware routing, responder workflows and offline synchronization.

Copy/Paste Prompt

```
## PHASE 3 - REAL ML INTEGRATION, ROUTING, RESPONDER AND OFFLINE SUPPORT
```

```
### ML integration
```

```
Keep StubMLProvider.
```

```
Add HttpMLProvider implementing internal-ml-openapi.yaml.
```

```
Required operations:
```

- nowcast
- inundation
- risk generation
- field-report verification
- model health/status

```
Backend must NEVER import PyTorch or model implementation code.
```

```
Add timeout, retry, circuit breaker and fallback-to-stub/demo behavior.
```

```
Never expose ML internal URL/token publicly.
```

```
### Field report verification
```

```
Complete workflow:
```

```
report -> object upload -> async ML verification -> backend policy -> verified / likely / human-review / rejected -> live event -> optional incident correlation.
```

```
Human override requires reason + audit event.
```

```
### Lower-risk routing
```

```
Implement RoutingProvider.
```

```
Prototype implementation: OSMnx / NetworkX.
```

```
Road edge cost:
```

```
travel cost + flood risk penalty + depth penalty + closure penalty + uncertainty penalty.
```

```
Hard-closed roads must be unusable.
```

```
API:
```

```
POST /api/v1/routes/lower-risk
```

```
Response:
```

```
route geometry
```

```
estimated cost
```

```
avoided risky segments
```

```
risk reasons
```

```
source/valid timestamp
```

```
Never label route "guaranteed safe".
```

```
### Field responder tasks
```

```
Implement:
```

```
VERIFY_LOCATION
```

```
MEASURE_DEPTH
```

```
ROAD_BLOCKED
```

DRAIN_BLOCKED
SUPPORT_REQUEST
ROAD_REOPENED
COMPLETE_TASK

Include assignee, geometry, version, evidence, status and incident relation.

Offline synchronization

Implement POST /api/v1/sync/batch.

client_id-based idempotency.

Responses:

accepted

duplicate

rejected

needs_merge

Use optimistic locking/base_version for responder updates.

Never silently overwrite conflict.

Tests

ML unavailable fallback

ML timeout

routing risk avoidance

road closure

report verification

offline duplicate mutation

optimistic conflict

region authorization

Stop after Phase 3.

Phase 3 credential checkpoint

- ML_SERVICE_URL and ML_SERVICE_TOKEN come from your own ML subteam deployment; they are internal secrets, not public API keys.
- OSMnx/NetworkX prototype routing needs no API key.
- Self-hosted OSRM/Valhalla also does not require a vendor API key; only configure its service URL.

PHASE 4 - Historical Replay, Response Optimization and Security Hardening

Goal: deterministic historical replay, explainable response-resource recommendations, complete auditability and security hardening.

Copy/Paste Prompt

```
## PHASE 4 - HISTORICAL REPLAY, RESPONSE OPTIMIZATION AND SECURITY HARDENING
```

```
### Historical replay
```

Create immutable replay manifest containing timestamped:

- weather
- model outputs
- risk snapshots
- reports
- incidents
- alerts

Implement virtual replay clock.

Live and replay data must never mix.

Support deterministic pause/play/scrub.

Precompute immutable replay tile manifests.

At replay end expose ML-provided predicted-vs-observed metrics.

```
### Model/data health
```

Build backend model/data health aggregation:

- source freshness
- missing percentage
- source quality
- latest model run
- model version
- data version
- inference duration
- degraded/fallback state

```
### Resource recommendation
```

Use Google OR-Tools.

Inputs:

- pumps
- teams
- shelters
- travel time
- capacities
- incident priorities
- zone risk

Output:

- suggested allocation plan
- objective score
- unfulfilled demand
- constraint violations.

Never auto-dispatch.

An authorized human converting a recommendation into tasks must create audit events.

Complete audit system

Append-only audit events for:

alert publication
report review/override
incident override
responder status
model activation
administrative changes
resource-plan approval.

Record:

actor
role
action
entity
timestamp
trace_id
before_hash
after_hash
supporting model/risk snapshot.

Security hardening

Implement:

- region-scoped RBAC
- strict server authorization
- rate limits
- payload size limits
- MIME validation
- signed uploads
- private object buckets
- secure response headers
- log redaction
- CORS allowlist
- no internal stack trace leakage
- secret filtering
- PII minimization

Add malware/content scanning integration interface, even if local implementation is stubbed.

Security tests

Attempt:

privilege escalation
cross-region access
expired token
malformed JWT
object URL abuse
file spoofing
oversized upload
rate-limit abuse
unauthorized alert publishing
responder task takeover.

Run complete regression and stop.

Phase 4 credential checkpoint

- Google OR-Tools is a library and needs no API key.
- Malware scanning should be behind an interface. A local ClamAV implementation can be used without a SaaS credential.
- Never make a production object bucket public. Use signed URLs and time-limited access.

PHASE 5 - Observability, Performance, CI and Release Hardening

Goal: end-to-end observability, load/performance validation, reproducible production-like Compose, CI and release documentation.

Copy/Paste Prompt

```
## PHASE 5 - OBSERVABILITY, PERFORMANCE, CI AND RELEASE HARDENING

### OpenTelemetry
Instrument:
FastAPI
SQLAlchemy/Postgres
Redis
Celery
ML HTTP calls
object-store calls
outbound notification calls.

Use one trace_id across:
API -> DB -> worker -> ML -> object store -> event -> notification.

### Prometheus
Export:
API p50/p95/p99
error rate
active requests
source lag
ingestion failures
Celery queue depth
ML inference latency
tile latency
notification failures
sync conflicts
DB pool state.

### Grafana
Add dashboards for:
API health
weather ingestion
ML/model operations
Redis/Celery
DB
realtime/WebSocket
alert delivery.

### Sentry
Integrate sentry-sdk for uncaught backend/worker failures.
DSN must come only from environment.
Never include auth tokens, citizen PII, images or secrets in Sentry breadcrumbs/context.

### Performance
Add:
```

Postgres query profiling
 GIST/index checks
 H3 indexes
 Redis caching
 ETags
 raster/vector cache headers
 pagination
 connection pooling
 async I/O.

Run load tests against:
 risk cells
 risk timeline
 tile manifests
 incidents
 WebSocket fan-out.

Production-like Compose

Compose must contain:

api
 worker
 scheduler
 postgres-postgis
 redis
 minio
 titiler
 pg_tileserv
 ml-stub or ml-service
 prometheus
 grafana

optional: redpanda

Add reverse proxy with TLS termination-ready config, HTTP -> HTTPS redirect production profile, API routing, tile routing and WebSocket upgrade support.

CI

Build pipeline:

lint -> type check -> unit tests -> contract tests -> migration-from-zero -> integration tests -> security tests -> Docker build -> vulnerability scan -> deploy-preview hook.

Final documentation

Generate:

README
 architecture
 local setup
 environment-variable reference
 API usage
 role matrix
 demo seed/replay steps
 troubleshooting
 deployment notes.
 No actual credentials in docs.

Run all final tests and produce a Definition-of-Done report.

Phase 5 credential checkpoint

- Prometheus and local Grafana need no external account. You choose the Grafana admin credentials.
- Sentry is optional. If enabled, obtain the DSN from the project settings and place it in SENTRY_DSN only.
- GitHub deployment secrets belong in Repository Settings -> Secrets and variables -> Actions, not in source files.

5. Recommended .env.example

IMPORTANT

Commit .env.example with placeholders only. Never commit .env. Real values belong in local/deployment secret storage.

```
# =====
# APP
# =====
APP_ENV=development
APP_NAME=jalrakshak-backend
APP_SECRET=CHANGE_ME
LOG_LEVEL=INFO
CORS_ORIGINS=http://localhost:3000

# =====
# POSTGRES / POSTGIS
# =====
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=jalrakshak
POSTGRES_USER=jalrakshak
POSTGRES_PASSWORD=CHANGE_ME
DATABASE_URL=postgresql+asyncpg://jalrakshak:CHANGE_ME@postgres:5432/jalrakshak

# =====
# REDIS / CELERY
# =====
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/1
CELERY_RESULT_BACKEND=redis://redis:6379/2

# =====
# OBJECT STORAGE
# =====
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=CHANGE_ME
MINIO_SECRET_KEY=CHANGE_ME
MINIO_BUCKET=jalrakshak
MINIO_SECURE=false

# =====
# AUTH
# =====
SUPABASE_URL=https://PROJECT_REF.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_REPLACE_ME

# Only if backend actually needs privileged Supabase admin operations
SUPABASE_SECRET_KEY=sb_secret_REPLACE_ME

# =====
# ML INTERNAL SERVICE
# =====
```

```

ML_SERVICE_URL=http://ml-stub:8001
ML_SERVICE_TOKEN=CHANGE_ME

# =====
# TILES
# =====
TITILER_URL=http://titiler:8000
PGTILESERV_URL=http://pg_tileserv:7800

# =====
# PUSH NOTIFICATIONS
# =====
EXPO_PUSH_ACCESS_TOKEN=

# =====
# OBSERVABILITY
# =====
SENTRY_DSN=
OTEL_SERVICE_NAME=jalrakshak-backend
OTEL_EXPORTER_OTLP_ENDPOINT=
PROMETHEUS_ENABLED=true

GRAFANA_ADMIN_USER=admin
GRAFANA_ADMIN_PASSWORD=CHANGE_ME

# =====
# EVENT STREAMING OPTIONAL
# =====
REDPANDA_BROKERS=
REDPANDA_USERNAME=
REDPANDA_PASSWORD=

```

6. Minimum Public API Contract

- GET /api/v1/weather/current
- GET /api/v1/weather/sources/status
- GET /api/v1/nowcast/manifest
- GET /api/v1/inundation/manifest
- GET /api/v1/risk/cells?bbox=&valid_time=
- GET /api/v1/risk/{h3_cell}/timeline
- GET /api/v1/incidents
- POST /api/v1/reports
- POST /api/v1/reports/{id}/review
- POST /api/v1/routes/lower-risk
- POST /api/v1/alerts/draft
- POST /api/v1/alerts/{id}/approve
- POST /api/v1/alerts/{id}/publish
- GET /api/v1/assets
- GET /api/v1/models/status
- POST /api/v1/sync/batch
- WS /api/v1/live

7. Minimum Realtime Event Topics

- risk.cell.updated
- incident.created
- incident.updated
- report.received
- report.verified
- alert.drafted
- alert.published
- source.degraded
- model.run.completed
- field.task.updated

8. Backend Definition of Done

- ☐ OpenAPI/contract generation
- ☐ PostgreSQL/PostGIS migrations from a fresh database
- ☐ Time-series risk/weather history
- ☐ S3/MinIO object layer
- ☐ Weather/source ingestion adapters
- ☐ STAC/manifest metadata
- ☐ TiTiler raster tiles
- ☐ pg_tileserv/vector tiles
- ☐ ML stub + real internal ML client
- ☐ Async workers and event bus
- ☐ OIDC auth + region-scoped RBAC
- ☐ Weather/risk/model APIs
- ☐ Incident lifecycle and append-only timeline
- ☐ Citizen field-report upload + verification orchestration
- ☐ Human review/override with audit
- ☐ Alert draft/approve/publish state machine
- ☐ CAP 1.2 serialization
- ☐ Push notification fan-out
- ☐ Watch locations
- ☐ Lower-risk routing
- ☐ Responder tasks and evidence
- ☐ OR-Tools response recommendation
- ☐ Historical replay clock/manifests
- ☐ Offline sync/conflict handling
- ☐ Append-only audit log
- ☐ Security/rate limits/signed URLs
- ☐ OpenTelemetry/Prometheus/Grafana/Sentry hooks
- ☐ Docker Compose full stack
- ☐ CI tests + seed/demo scripts

9. Final Rules for the AI Coding Assistant

- Never implement all phases in one uncontrolled pass.
- Never remove stubs when a real provider is added; stubs are essential for independent development and demos.
- Never let web/mobile bypass the backend to call ML, Postgres, MinIO or internal services.
- Never expose internal storage paths, credentials or stack traces in public APIs.
- Never claim a lower-risk route is guaranteed safe.
- Never auto-publish high-impact alerts; preserve analyst/approver authorization.
- Never claim the prototype directly publishes to NDMA SACHET without authorized integration.
- Never auto-dispatch emergency resources from OR-Tools output; recommendations require human approval.
- Never overwrite raw weather source artifacts; preserve checksum/provenance.
- Never silently overwrite offline conflicts.
- Never commit secrets.

FINAL IMPLEMENTATION FLOW

Master Context -> Phase 0 -> verify -> Phase 1 -> verify -> Phase 2 -> verify -> Phase 3 -> verify -> Phase 4 -> verify -> Phase 5 -> complete Definition-of-Done report. This sequencing lets Backend, ML, Web and Mobile teams develop independently and integrate without rewriting core modules.